

Laboratorio di Algoritmi e Strutture Dati

Relazione del progetto "Automi e segnali"

Federico Andrea Rizzi 43201A

1 Introduzione

In questo documento vengono presentate le scelte fatte in merito all'approccio, alla modellazione e alla risoluzione del problema sia per quanto concerne le strutture dati utilizzate che agli algoritmi implementati. Il file **zip** consegnato contiene:

- il file `.go` con nome "43201A_rizzi_federico_andrea.go" contenente l'intero codice sorgente
- la relazione (questo file) in formato **pdf** con nome "43201A_rizzi_federico_andrea.pdf"
- TODO: test

2 Considerazioni sul problema

Le entità principali da modellare sul piano (oltre al piano stesso) sono gli **automati** e gli **ostacoli**. Gli aspetti fondamentali sono:

- ogni automa ha un nome univoco sull'alfabeto $\{0, 1\}$ e si trova in un punto del piano
- su un punto (in un dato momento) possono giacere o un insieme di automi o un insieme di ostacoli
- non è detto che su ogni punto del piano (i quali punti sono infiniti) sia presente o un automa o un ostacolo

Di cruciale importanza è la richiesta di sapere, dato un punto (x, y) e il nome η di un automa, se esiste un percorso libero da $P(\eta)$ a (x, y) di lunghezza $D(P(\eta), (x, y))$. E' la richiesta più complicata da risolvere (in quanto i richiami si basano su di essa) e si possono fare le seguenti considerazioni:

- poichè si richiede l'esistenza di un percorso libero con lunghezza pari alla lunghezza minima di un percorso tra i due punti, i passi unitari possono essere fatti seguendo al massimo 2 direzioni sulle 4 possibili. Per esempio, un percorso di distanza minima tra il punto $(2, 2)$ e il punto $(5, 5)$ può essere composto, partendo da $(2, 2)$, solo da passi unitari in cui si incrementa di 1 la coordinata x o si incrementa di 1 la coordinata y (non è mai possibile decrementare tali coordinate, o non si avrebbe un percorso di distanza minima)
- partendo da un automa è sicuro che il punto di partenza non si trovi all'interno dell'area di un ostacolo. Nella simulazione di un percorso tra un automa e un punto possiamo affermare che tale percorso non è libero non appena incontra un punto che si trova sul perimetro di un ostacolo. Può essere quindi interessante memorizzare i punti che si trovano sul perimetro degli ostacoli

3 Modellazione e strutture dati utilizzate

Sulla base delle considerazioni fatte e delle operazioni da realizzare è possibile scartare a priori alcune modellazioni possibili delle entità del problema. In particolare si può considerare inefficiente la rappresentazione del piano attraverso una matrice che contenga una corrispondenza tra un punto e l'entità presente correntemente sul piano in tale punto (un automa, un ostacolo o nulla). Questo perchè il

piano è potenzialmente infinito e la minima porzione di esso che contiene tutti gli automi e gli ostacoli presenti può essere molto grande. Inoltre per rappresentare un possibile piano con un automa nel punto $(-1000, -1000)$ e un automa nel punto $(1000, 1000)$ verrebbe richiesta una matrice 2001×20001 , il che è inefficiente in termini di spazio.

Per quanto riguarda gli ostacoli non è necessario memorizzare tutti i punti all'interno della loro area, in quanto pochissime operazioni gioverebbero di tale scelta (solo l'operazione di *stato* dato un punto, che troverebbe, se tale punto è interno all'ostacolo, risposta in tempo costante). In particolare un ostacolo è stato rappresentato in questo modo:

```
1 type punto struct {
2     x int
3     y int
4 }
5 type ostacolo struct {
6     bassoSx punto
7     altoDx punto
8 }
```

La struttura *punto* contiene le coordinate x ed y di un punto del piano, mentre la struttura *ostacolo* contiene le coordinate del vertice in basso a sinistra e del vertice in alto a destra di un ostacolo. Dato un ostacolo R e un punto P la richiesta di sapere se P è contenuto in R viene fatta in tempo $O(1)$.

Per quanto riguarda l'insieme di ostacoli presenti in un piano è stato definito un tipo di dato astratto:

```
1 type ListOstacoli interface {
2     AddOstacolo(r ostacolo)
3     ContienePunto(p punto) bool
4     StampaOstacoli()
5 }
```

Tale tipo di dati rappresenta una lista di ostacoli in grado di eseguire le seguenti operazioni:

- **AddOstacolo**: dato un ostacolo come parametro, lo aggiunge alla lista
- **ContienePunto**: dato un punto come parametro, restituisce *true* se tale punto è contenuto in almeno un ostacolo della lista, *false* altrimenti
- **StampaOstacoli**: stampa gli ostacoli presenti nella lista

Queste operazioni sono tutte e sole quelle che un insieme di ostacoli deve saper fare per rispondere al problema (per esempio, non è richiesto che un ostacolo possa essere eliminato). Tale tipo di dato astratto è stato implementato mediante l'utilizzo di una lista concatenata, con puntatore alla testa e alla coda. Questo permette di avere operazioni di inserimento in tempo $O(1)$. L'unico punto a sfavore è che le altre due operazioni, dato n il numero di ostacoli, costano $O(n)$. Ho cercato di trovare un ordinamento possibile per gli ostacoli, in base a qualche tipo di parametro (ad esempio coordinate o distanza rispetto a un punto dato) per avere una ricerca logaritmica rispetto al numero di ostacoli ma non sono riuscito. Potrebbe essere un miglioramento possibile, ma l'inserimento avrebbe costo $O(n)$.

A questo punto, per modellare l'intero piano, con tutte le entità del problema, è stata scelta la seguente soluzione:

```
1 type piano *struct {
2     automi map[string]punto
3     ostacoli ListOstacoli
4     puntiConosciuti map[punto]string
5 }
```

Innanzitutto il piano è rappresentato da un puntatore ad una struttura, questo per facilitare il passaggio per parametro di esso a funzioni (in quanto in Go le strutture verrebbero passate per copia come parametro), e le eventuali modifiche saranno riflesse su di esso. Gli automi sono stati rappresentati

con una mappa che ha per chiave una stringa che rappresenta il loro nome univoco, e il valore è un punto che rappresenta la loro posizione nel piano. Questo permette, dato il nome di un automa, di avere accesso alla sua posizione in tempo costante, e tale funzionalità è utile a molte operazioni (come l'esistenza di un percorso). Gli ostacoli sono rappresentati dal tipo di dato astratto *ListOstacoli* presentato precedentemente, che verrà implementato tramite una lista concatenata, come spiegato prima. Ogni nodo della lista concatenata rappresenta un ostacolo presente sul piano. Da notare che, nel caso venisse trovato un possibile ordinamento degli ostacoli per avere delle ricerche logaritmiche in base al numero di essi, basterebbe implementare l'interfaccia con un tipo di dati concreto e inizializzare il piano utilizzando tale tipo, tutte le operazioni continuerebbero a funzionare.

Il piano viene rappresentato utilizzando una struttura dati aggiuntiva oltre a quelle per modellare automi e ostacoli. Essa ha per nome **puntiConosciuti** ed è una mappa che ha per chiave un punto e per valore una stringa. L'intento di tale struttura è memorizzare i punti conosciuti del piano di cui si conosce l'entità che giace su di essi, in modo tale da sfruttare tali informazioni per implementare le operazioni richieste. In particolare:

- Quando viene aggiunto un automa in un punto del piano, bisognerà aggiornare tale struttura dati. Dato un punto del piano, se su di esso giace uno o più automi, allora la chiave rappresentata da tale punto, nella struttura, avrà per valore Ax , dove x indica il numero di automi giacenti su tale punto
- Quando viene aggiunto un ostacolo, la struttura conterrà tutti una entry per ogni punto del perimetro di tale ostacolo. In particolare tutti i punti del perimetro dell'ostacolo saranno memorizzati nella struttura e avranno per valore la stringa "O". Questo permette di avere accesso ai punti del perimetro degli ostacoli in tempo $O(1)$. Come detto in precedenza, se viene cercata l'esistenza di un percorso libero con distanza minima tra un automa e un punto, è certo che nella simulazione di un percorso, se esso va contro un punto del perimetro di un ostacolo, si può dire che non è libero. Sicuramente un percorso andrà prima a "sbattere" contro un punto di un perimetro di un ostacolo che su un punto interno all'area di un ostacolo
- Quando viene richiesto lo stato di un punto del piano e si scopre che esso è vuoto, nella struttura verrà memorizzata tale informazione. Quindi se viene richiesto lo stato del punto $(5, 3)$ del piano e su di esso non giace nè un automa nè un ostacolo, la struttura avrà una entry con chiave il punto $(5, 3)$ e valore la stringa "E"

Viene introdotta della ridondanza di informazioni, poichè dato il vertice in basso a sinistra e il vertice in alto a destra di un ostacolo è possibile sapere tutti i punti compresi nell'area di tale ostacolo. Però, per sapere tali informazioni per ogni ostacolo il tempo impiegato è proporzionale al numero di ostacoli, mentre con l'introduzione di questa struttura, per i punti di cui è interessante sapere se giacciono su un ostacolo, il tempo impiegato è costante (questo ridurrà moltissimo il tempo richiesto per l'operazione di esistenza di un percorso tra un punto e un automa e quindi anche quella di *richiamo*). Anche gli automi sono rappresentati in maniera ridondante, ma questo permette, di sapere in tempo costante se in un punto del piano giace un automa (e questo è richiesto per esempio dall'operazione di *stato*), senza dover scorrere tutti gli automi presenti.

Inoltre se un utente richiede lo stato di moltissimi punti del piano, ed essi sono vuoti (non giace nessuna entità) la struttura memorizzerà tutte queste informazioni. Quello che ci si aspetta, normalmente, è che un utente richieda lo stato di un punto del piano per sapere se può effettuare l'inserimento di un automa. La memorizzazione del fatto che un punto sia vuoto serve proprio a questo: quando poi l'utente richiederà l'inserimento di un automa non bisognerà ricontrollare se il punto scelto per esso giace su degli ostacoli, ma si può sapere in tempo costante (se si scopre che gli utenti richiedono lo stato di molti punti senza la finalità di aggiungere un automa o un ostacolo giacente su di essi sarà più utile non memorizzare lo stato di punti vuoti). Naturalmente queste informazioni necessitano di essere coerenti tra di loro.

4 Operazioni e costi

In seguito verrà discussa l'implementazione degli algoritmi per risolvere le operazioni richieste, con i relativi costi.

4.1 stampa

Questa operazione viene implementata dall'omonima funzione e si occupa di stampare l'elenco degli automi seguito dall'elenco degli ostacoli secondo il formato specificato nelle specifiche di implementazione. A tal proposito è necessario iterare tutti gli elementi della mappa `automi` e stamparne nome e coordinate tramite una funzione `Println` del package `fmt`, che si può assumere abbia costo $O(1)$ in quanto generalmente la stringa da stampare è molto corta (verrà fatta tale assunzione ogni volta che è presente una funzione di stampa). Inoltre è necessario stampare le coordinate di tutti gli ostacoli, e quindi utilizzare il metodo `StampaOstacoli`, il quale dovrà scorrere tutti gli ostacoli presenti in quanto la lista di ostacoli è una lista concatenata.

Tempo: il tempo necessario per eseguire tale funzione, dato n il numero di automi nel piano e m il numero di ostacoli è $O(n + m)$.

Spazio: non è previsto l'uso di spazio aggiuntivo rispetto al piano passato come parametro, quindi lo spazio usato è $O(1)$.

4.2 ostacolo

Questa operazione, specificati i vertici in basso a sinistra e in alto a destra, permette di creare un ostacolo sul piano se esso non contiene punti su cui giacciono automi. Questa operazione è implementata dalla funzione `aggiungiOstacolo` e quello che occorre fare è:

- Iterare tutti gli automi nel piano (attraverso la mappa che li rappresenta) e verificare che nessun automa giace sull'area su cui si vuole posizionare l'ostacolo. Data la posizione di un automa questa verifica si fa in tempo costante grazie alla funzione `contiene`. Il costo è quindi relativo all'iterazione degli automi nel piano, quindi sia a il numero di automi il costo in termini di tempo di questa parte è $O(a)$
- Se l'area in cui si vuole collocare l'ostacolo non contiene alcun automa, bisogna aggiungere l'ostacolo alla lista di ostacoli del piano. Questo inserimento è fatto in tempo $O(1)$ per via dell'uso di una lista concatenata con puntatore alla coda
- Una volta aggiunto l'ostacolo alla lista, per mantenere la coerenza su quanto detto sulla struttura `puntiConosciuti` del piano, è necessario aggiungere i punti del perimetro di tale ostacolo ai punti conosciuti del piano, marcandoli come punti su cui giace un ostacolo. Siano $P = (x_0, y_0)$ e $Q = (x_1, y_1)$ rispettivamente il vertice in basso a sinistra e il vertice in alto a destra dell'ostacolo, allora si definisce $n = x_1 - x_0$ e $m = y_1 - y_0$ e il costo in termini di tempo di quanto detto è quindi $O(n + m)$

Tempo: in base a quanto spiegato il costo in termini di tempo di questa operazione è $O(a + n + m)$.

Spazio: vengono usate variabili di supporto per creare due punti e un rettangolo. Il costo in spazio è $O(1)$.

4.3 posizioni

Questa operazione, specificato un prefisso, si occupa di stampare la posizione degli automi nel piano con prefisso dato. Tale operazione viene implementata dall'omonima funzione e prevede lo scorrimento di tutti gli automi nel piano, grazie alla struttura `automi` del piano, e per ogni automa viene controllato se ha il nome con prefisso dato. Tale verifica viene fatta grazie alla funzione `HasPrefix` del package `strings` il cui costo in termini di tempo è dato dalla lunghezza del prefisso nel caso peggiore.

Tempo: sia a il numero di automi presenti nel piano e n la lunghezza del prefisso, allora il costo di questa operazione in termini di tempo è $O(a \cdot n)$.

Spazio: non viene utilizzato spazio aggiuntivo, quindi il costo è $O(1)$.

4.4 stato

Tale operazione, date le coordinate di un punto del piano, si occupa di stampare "A" se sul punto giace un automa, "O" se sul punto giace un ostacolo "E" altrimenti. Questa operazione viene implementata dall'omonima funzione e viene controllato se nella struttura `puntiConosciuti` è presente una entry che ha per chiave il punto con coordinate specificate dall'utente. Se questo è vero la risposta viene data in tempo $O(1)$. Generalmente questo succede quando la richiesta viene fatta su un punto su cui giace un automa o il perimetro di un ostacolo.

Se tale entry non è presente, allora bisogna iterare su tutti gli ostacoli presenti e capire se almeno uno di tali ostacoli contiene il punto specificato.

Tempo: nel caso peggiore bisogna, come detto prima, iterare su tutti gli ostacoli. Sia n il numero di ostacoli, il costo in termini di tempo di questa operazione è $O(n)$.

Spazio: vengono utilizzate due variabili aggiuntive di supporto, il costo è $O(1)$.

4.5 automa

Questa operazione, specificato il nome di un automa e il punto su cui posizionarlo, lo posiziona nel piano se in tale punto non giace già un ostacolo (se l'automa esiste già viene riposizionato). Essa viene implementata dall'omonima funzione e viene controllato se si conosce se nel piano in tale punto è presente un automa o non giace nulla (tramite la struttura `puntiConosciuti`). Se è questo il caso vengono aggiornate tutte le strutture per mantenere la coerenza di dati e posizionato l'automa nel punto. Invece se si sa che su tale punto giace un ostacolo non viene fatto nulla. Tutte le funzioni e istruzioni eseguite fino a questo punto costano tempo $O(1)$. Nel caso in cui non si conosce se giace o meno un automa, un ostacolo o nulla su tale punto, è necessario iterare tutti gli ostacoli e controllare se almeno uno di essi contiene il punto su cui si vuole posizionare l'automa.

Tempo: il tempo nel caso peggiore, dato n il numero di ostacoli è $O(n)$ ed esso si presenta quando non si conosce tramite la struttura `puntiConosciuti` se sul punto specificato giace o meno un automa o un ostacolo.

Spazio: vengono usate delle variabili di supporto, ma esse sono in numero sempre uguale e occupano lo stesso spazio indipendentemente dalla lunghezza dell'istanza in input. Il costo in spazio è quindi $O(1)$.

4.6 esistePercorso

Questa operazione, dato un punto di arrivo (x, y) nel piano e il nome η di un automa, richiede di sapere se è possibile raggiungere tale punto di arrivo partendo da $P(\eta)$ attraverso un percorso libero di lunghezza $D(P(\eta), (x, y))$. Tale distanza rappresenta la lunghezza minima dei percorsi che collegano i due punti. Innanzitutto è possibile vedere il piano come un grafo non orientato in cui:

- i vertici rappresentano i punti del piano (insieme V)
- gli archi rappresentano un qualsiasi passo unitario ($E \subseteq V \times V$). Quindi $(P, Q) \in E$ se e solo se P e Q sono collegati da un passo unitario (sia orizzontale che verticale)

Tale grafo avrebbe quindi l'insieme V dei vertici ed E degli archi con cardinalità infinita, dunque non avrebbe alcun senso tenerlo memorizzato. E' possibile rappresentarlo implicitamente poichè si ricorre ad esso solamente quando viene richiesta tale operazione. In tal caso il vertice di partenza sarà quello che rappresenta il punto $P(\eta)$ e il vertice di arrivo sarà quello che rappresenta il punto (x, y) di arrivo. Poichè i passi unitari son ben definiti è rappresentare implicitamente anche gli archi del grafo in quanto è noto se esiste un arco tra due vertici (esiste solo se i punti rappresentati dai due vertici sono collegati attraverso un passo unitario nel piano). Il costo in spazio di rappresentazione di tale grafo, dunque, è $O(1)$ in quanto serve solamente tenere in memoria i vertici che rappresentano il punto di partenza e di arrivo.

Inoltre, come già detto nelle considerazioni iniziali, poichè siamo alla ricerca di un percorso libero con lunghezza minima, ci si può muovere attraverso al massimo 2 delle 4 possibili direzioni, ed esse possono essere calcolate facilmente attraverso le coordinate dei due punti.

4.6.1 Primo possibile approccio

Un primo approccio al problema consiste nell'utilizzare un algoritmo di visita in ampiezza (BFS) o in profondità (DFS). Tali algoritmi, partendo dal vertice che rappresenta il punto di arrivo, simulano tutti i possibili percorsi (con due strategie differenti) nelle direzioni possibili (che sono 1 o 2). La BFS ad ogni iterazione visita i vicini del vertice corrente, e tali vicini saranno i vertici raggiungibili con un solo arco, ovvero i punti raggiungibili da un passo unitario nelle, al massimo, 2 direzioni possibili. La DFS, diversamente dalla BFS, ad ogni iterazione cerca di avanzare il più lontano possibile lungo una direzione. Questo significa che il percorso verrà esteso in una direzione scelta tra quelle possibili finché si incontrano ostacoli o si raggiunge il punto di arrivo (x, y) . Una volta esaurita una possibile direzione, si ritorna indietro al punto più recente in cui erano rimaste altre direzioni da esplorare, continuando la ricerca da lì.

Entrambe le visite terminano la simulazione di un possibile percorso quando viene visitato un vertice che rappresenta un punto del piano su cui giace il perimetro di un ostacolo. E' sicuro che, se un percorso incontra un ostacolo, lo fa inizialmente su un punto del perimetro di tale ostacolo, poichè un automa sicuramente non giace su punti appartenenti all'area di un ostacolo. Quindi questa verifica viene fatta in tempo $O(1)$, ed è il vantaggio di tenere in memoria la struttura `puntiConosciuti` che memorizza i punti del perimetro degli ostacoli. Se non ci fosse tale struttura, per ogni punto visitato bisognerebbe scorrere tutta la lista di ostacoli, che ha costo lineare rispetto al numero di essi nel piano. Entrambe le visite terminano quando vengono esplorati tutti i possibili percorsi di lunghezza $D(P(\eta), (x, y))$, e se esiste un percorso libero di tale lunghezza, sarà stato visitato il vertice che rappresenta il punto di arrivo.

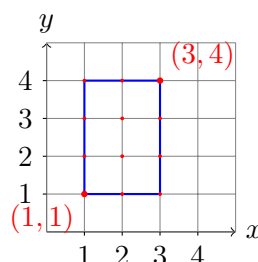
Questo approccio è efficace ma **non efficiente**. Questo perchè entrambe le visite, nel caso peggiore, simulano tutti i possibili percorsi di lunghezza minima tra $P(\eta)$ e (x, y) , i quali, se le direzioni percorribili sono 2 (quindi i punti di arrivo e partenza non si trovano o sulla stessa ascissa o sulla stessa ordinata), sono 2^d dove $d = D(P(\eta), (x, y))$. Tali algoritmi lavorano quindi in tempo esponenziale rispetto alla distanza D tra i due punti, in quanto simulano una ricerca esaustiva di un percorso libero e sono quindi impraticabili.

4.6.2 Approccio usato

Per risolvere il problema è stato usato un approccio di programmazione dinamica. Gli approcci precedenti simulavano tutti i possibili percorsi di lunghezza minima tra il punto di partenza e il punto di arrivo. E' possibile però fare delle considerazioni su tale problema, e astrarne i concetti fondamentali:

- un percorso libero di lunghezza minima è formato da sottopercorsi liberi di lunghezza minima (principio di ottimalità)
- non è tanto importante la simulazione del percorso, ma capire se questo esiste. E' possibile individuare l'insieme di punti che un percorso libero di distanza minima potrebbe percorrere e chiedersi per ognuno di tali punti una volta per tutte se esiste un percorso libero di distanza minima tra esso e il punto di partenza. Questo permette di risolvere il problema per tale punto una sola volta, senza simulare tutti i percorsi che passano per tale punto

Ad esempio, se viene cercata l'esistenza di un percorso tra il punto $(1, 1)$ e il punto $(3, 4)$ di distanza $D((1, 1), (3, 4))$ è possibile individuare un insieme di punti candidati ad essere percorsi:



Solo i punti appartenenti all'area della figura in blu possono essere percorsi, altrimenti il percorso non sarebbe di lunghezza minima. Tale figura può essere vista come una matrice di programmazione dinamica (che sarà una matrice di `bool`), e in generale tale matrice dato il punto di partenza $P = (x_0, y_0)$ e il punto di arrivo $Q = (x_1, y_1)$ ha dimensione $n \times m$ dove $n = |y_1 - y_0| + 1$ e $m = |x_1 - x_0| + 1$. Per risolvere il problema dell'esistenza di un percorso libero di lunghezza minima tra i due punti deve essere percorsa la matrice secondo delle direzioni precise e partendo da entrate precise di essa. Questo dipende dalle direzioni percorribili tra i due punti per avere un percorso libero di lunghezza minima. In questo caso è possibile, partendo da $(1, 1)$ fare passi unitari orizzontali che aumentano di 1 la coordinata x e fare passi unitari verticali che aumentano di 1 la coordinata y di un punto. Quindi la matrice deve essere popolata a partire dall'entrata $(0, 0)$ esaminando riga per riga (aumentandone l'indice) le possibili colonne dall'indice più basso a quello più alto.

Se fosse stato richiesto un percorso con punto di partenza $(3, 4)$ e punto di arrivo $(1, 1)$, i passi unitari orizzontali possibili sarebbero stati quelli che diminuiscono di 1 la coordinata x e i passi unitari orizzontali possibili sarebbero stati quelli che diminuiscono di 1 la coordinata y di un punto. Quindi la matrice dovrebbe essere popolata dall'entrata $(3, 2)$, esaminando riga per riga (diminuendone l'indice) le possibili colonne dall'indice più alto a quello più basso.

Dato l'alto numero di direzioni possibili per popolare la matrice di programmazione dinamica è possibile fare una considerazione. Se esiste un percorso libero di lunghezza minima tra un punto di partenza e un punto di arrivo, esiste anche se si parte dal punto di arrivo. Questo permette di decidere di partire dal punto, tra i due, che permette di popolare la matrice seguendo un ordine crescente di colonne (quindi tale per cui i passi orizzontali sono fatti aumentando di 1 la coordinata x). Quindi, se viene richiesta l'esistenza di un percorso libero tra il punto $(3, 4)$ e il punto $(1, 1)$ è possibile comunque partire dal punto $(1, 1)$. Questo fa sì che il punto di partenza per popolare la matrice e l'ordine da seguire dipenda solo dalla direzione dei passi unitari verticali. Se essi possono essere effettuati aumentandone la coordinata y si parte dall'entrata $(0, 0)$ della matrice e si popola essa esaminando riga per riga, aumentandone l'indice, le possibili colonne. Al contrario si popola essa esaminando riga per riga, diminuendone l'indice, le possibili colonne. Le colonne vengono esaminate sempre dall'indice più basso a quello più alto.

Popolare la matrice:

A questo punto è possibile descrivere come viene popolata la matrice di programmazione dinamica, DP da ora. Come accennato precedentemente è una matrice di `bool`, e dato il punto di partenza, il punto di arrivo e la direzione dei passi unitari verticali è possibile far corrispondere un'entrata di tale matrice ad un punto del piano in modo univoco. Il valore dell'entrata $DP[i][j]$ della matrice ha il seguente significato, una volta popolata:

- **true** se esiste un percorso libero di distanza minima dal punto di partenza designato al punto rappresentato dall'entrata della matrice
- **false** altrimenti

Essa, come detto precedentemente, viene popolata a partire dalla colonna 0 e da una riga dipendente dai passi unitari verticali del percorso. Si supponga che tali passi unitari verticali siano tali che venga aumentata di 1 la coordinata y dei punti intermedi. Quindi la matrice viene popolata riga per riga (da indice 0 all'ultimo indice disponibile), scorrendo le colonne dall'indice 0 all'ultimo indice disponibile. Inizialmente viene inizializzata l'entrata di partenza della matrice a **true** (in questo caso l'entrata $(0, 0)$). Tale valore può cambiare se e solo se il punto rappresentato da tale entrata si trova su un perimetro di un ostacolo, e in tal caso viene cambiato a **false**. In tutti gli altri casi rimane invariato. A questo punto vengono esaminate le altre entrate della matrice, e per ogni entrata (i, j) :

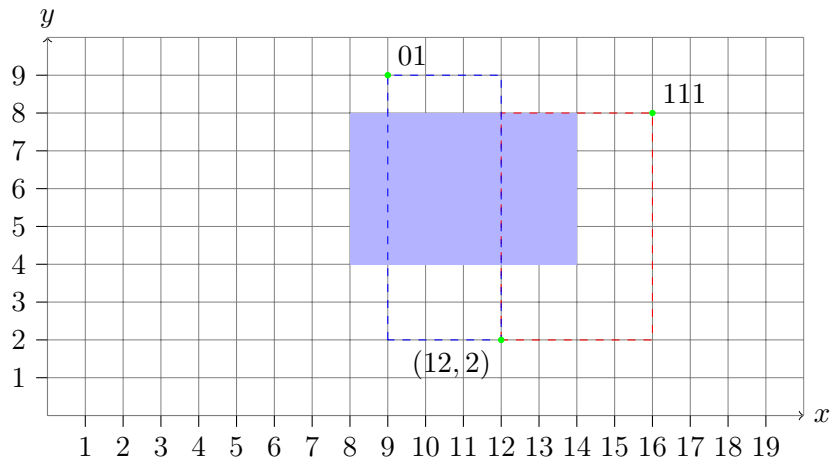
- Se il punto del piano rappresentato da tale entrata si trova sul perimetro di un ostacolo $DP[i][j]$ avrà valore **false**
- Se ciò non accade vengono controllate le entrate $(i, j - 1)$ e $(i - 1, j)$, se esistono. Se almeno una di essa ha valore **true** allora $DP[i, j]$ avrà valore **true**, altrimenti avrà valore **false**

Esaminare le due entrate descritte vuol dire esaminare i punti che si trovano esattamente a un passo unitario orizzontale e verticale di distanza dal punto rappresentato dall'entrata della matrice. Se almeno uno di essi è raggiungibile da un percorso libero di distanza minima a partire dal punto di partenza, allora lo è anche il punto in esame. Se la direzione dei passi unitari verticali avviene diminuendo di 1 la coordinata y dei punti esaminati, al posto che l'entrata $(i-1, j)$ verrà controllata l'entrata $(i+1, j)$ e la matrice verrà popolata nell'ordine descritto precedentemente.

Alla fine di tale operazioni la matrice nell'entrata che rappresenta il punto di arrivo avrà valore **true** se esiste un percorso libero di distanza minima tra i due punti specificati all'inizio, **false** altrimenti.

Alcuni esempi:

Si consideri un piano con un rettangolo $R(8, 4, 14, 8)$, un automa con nome 111 in posizione (16, 8) e un automa con nome 01 in posizione (9, 9). E' possibile chiedersi se esiste un percorso libero tra l'automa 111 e il punto (12, 2). Per quando detto in precedenza si può partire dal punto (12, 2) e arrivare al punto (16, 8). La matrice verrà popolata dall'entrata (0, 0) fino all'entrata (6, 4).



La matrice risultante sarà la seguente (dove 1 sta per **true** e 0 per **false** e la riga 0 è quella più in alto):

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

L'entrata (6, 4) risulta **true** e quindi esiste un percorso libero di distanza minima tra i due punti. Invece, se ci si chiede dell'esistenza di un percorso tra l'automa 01 e il punto (12, 2), il punto di partenza resterà quello su cui giace 01 e la matrice verrà popolata dall'entrata (8, 0) fino all'entrata (0, 3) poichè la direzione dei percorsi unitari verticali è verso il basso. La matrice risultante sarà la seguente:

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

L'entrata (0, 3), la quale rappresenta il punto (12, 2) risulta **false** e quindi non esiste un percorso con le condizioni previste.

Tempo: il tempo per risolvere questo problema è proporzionale alla dimensione della tabella. Sia $P = (x_0, y_0)$ il punto di partenza e $Q = (x_1, y_1)$ il punto di arrivo, allora il costo in termini di tempo è dato da $O(r \cdot c)$ dove $r = |y_1 - y_0| + 1$ e $c = |x_1 - x_0| + 1$. Tutte le funzioni chiamate all'interno della funzione `esistePercorso` vengono eseguite in tempo costante. Questo è possibile grazie alla mappa `puntiConosciuti` del piano che permette di avere accesso ai punti del perimetro degli ostacoli in tempo costante. Senza questa struttura per ogni entrata della matrice bisognerebbe iterare su tutti gli ostacoli per sapere se il punto rappresentato da tale entrata è contenuto in un ostacolo.

Spazio: lo spazio richiesto è relativo alla matrice di programmazione dinamica e anch'esso è $O(r \cdot c)$.

4.7 richiamo

Tale operazione si compone di varie parti (ed è implementata dall'omonima funzione nel programma). Dato il punto di richiamo Q e la stringa s :

- è necessario prima controllare se su Q giace un ostacolo. Questo viene fatto in tempo $O(1)$ se tale punto giace sul perimetro di un ostacolo, in tempo $O(n)$ altrimenti (dove n è il numero di ostacoli presenti nel piano). Se ciò accade è possibile rispondere al problema, altrimenti si deve andare avanti
- è poi necessario individuare gli automi che hanno nome con prefisso s . Questo prevede di iterare su tutto il numero di automi e utilizzare la funzione `HasPrefix` del package `strings` e aggiungere gli automi che soddisfano la condizione precedente in una slice. Sia a il numero di automi e l la lunghezza della stringa s , questo viene fatto in tempo $O(l \cdot a)$ poichè l'aggiunta di un elemento ad una slice viene fatto in tempo $O(1)$ ammortizzato
- A questo punto, per ogni automa individuato nel punto precedente è necessario chiedersi se esiste un percorso con lunghezza minima tra esso e il punto di richiamo. A tal proposito viene utilizzata la funzione `esistePercorso` spiegata precedentemente. Viene mantenuta una variabile `min` per sapere quale è la distanza minima tra tutti gli automi presi in considerazione e per cui esiste un percorso verso il punto di richiamo. In questo modo si può evitare di dover invocare la funzione `esistePercorso` inutilmente. In ogni caso, nel caso peggiore la distanza dal punto di richiamo e il punto su cui giace ogni automa che deve rispondere al richiamo è uguale. A questo punto è necessario eseguire la funzione di esistenza di un percorso per ogni automa, e questo per ognuno di esso ha un costo $O(r \cdot c)$ (tali parametri hanno lo stesso significato di quanto spiegato nella sezione precedente). Poichè a è il numero di automi presenti nel piano, nel caso peggiore tutti devono rispondere al richiamo, portando questa parte a costare $O(a \cdot r \cdot c)$

Tempo: il tempo totale è quindi $O(n + l \cdot a + a \cdot r \cdot c)$. In situazioni naturali n e $l \cdot a$ sono trascurabili rispetto a $a \cdot r \cdot c$, portando il costo a $O(a \cdot r \cdot c)$.

Spazio: viene usata una slice per contenere gli automi che devono rispondere al richiamo, una slice per contenere gli automi che effettivamente dovranno spostarsi sul punto di richiamo e ad ogni chiamata di `esistePercorso` viene allocata la matrice di programmazione dinamica (che viene deallocata quando termina l'esecuzione di tale funzione). Il costo in termini di spazio è quindi $O(a + r \cdot c)$.